

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I U.Divya/192372277 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Divya

Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

Aim

To implement password hashing using a salt and repeated hashing to demonstrate the concept of salting and key stretching.

Algorithm

1. Read the password from the user.
2. Generate a random salt.
3. Combine the password and salt.
4. Apply a simple hash repeatedly to simulate key stretching.
5. Store the resulting hash and salt.

C Program

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <time.h>

#define ITERATIONS 100000

unsigned long hash_function(char *password, unsigned int salt)
{
    unsigned long hash = 5381;
    int i;
    for (i = 0; password[i] != '\0'; i++)
        hash = ((hash << 5) + hash) + password[i];
```

```

    hash ^= salt;

    return hash;
}

unsigned long key_stretch(char *password, unsigned int salt)
{
    unsigned long hash;

    int i;

    hash = hash_function(password, salt);

    for (i = 0; i < ITERATIONS; i++)
    {
        hash = hash * 33 + i;

        hash ^= salt;
    }

    return hash;
}

int main()
{
    char password[100];

    char verify[100];

    unsigned int salt;

    unsigned long stored_hash;

    unsigned long verify_hash;

    srand(((unsigned int)time(NULL)));

```

```
printf("PASSWORD HASHING WITH SALTING\n");

printf("-----\n");

printf("Enter Password: ");

scanf("%99s", password);

/* Generate random salt */

salt = rand();

/* Generate password hash */

stored_hash = key_stretch(password, salt);

printf("\nGenerated Salt: %u", salt);

printf("\nStored Password Hash: %lu", stored_hash);

printf("\n\nEnter Password for Verification: ");

scanf("%99s", verify);

/* Generate hash for verification */

verify_hash = key_stretch(verify, salt);

if (stored_hash == verify_hash)

    printf("\nPassword Verification: SUCCESS\n");

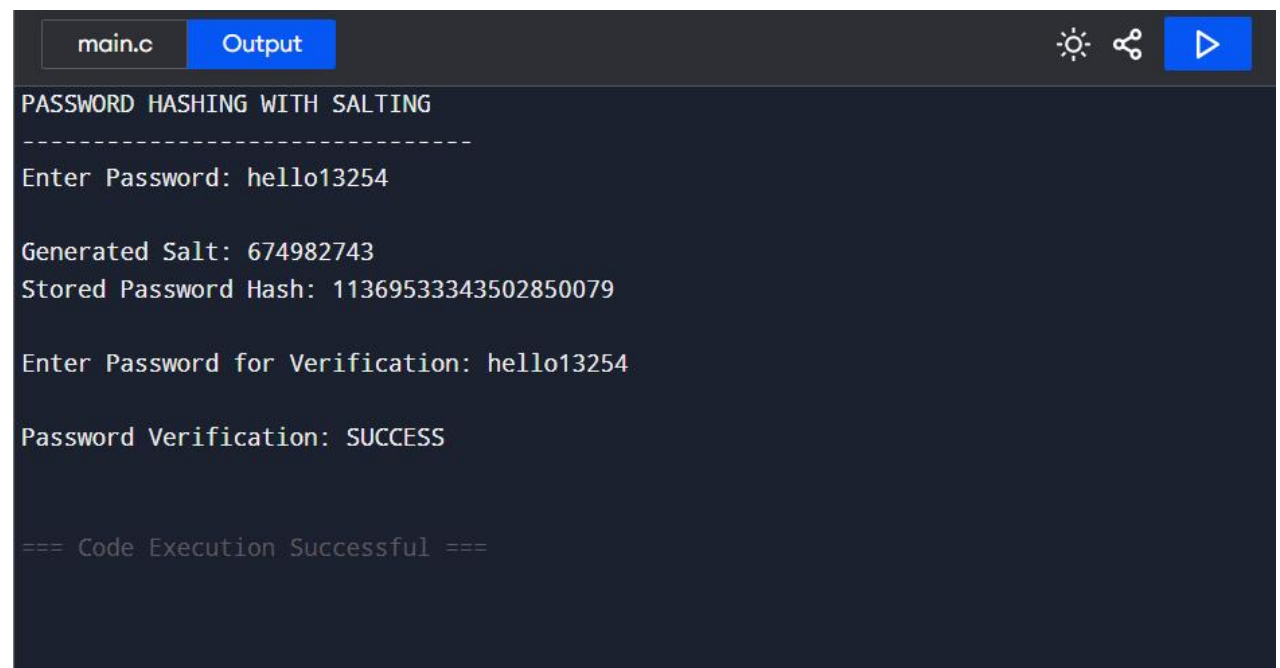
else

    printf("\nPassword Verification: FAILED\n");

return 0;

}
```

Output



```
main.c Output
PASSWORD HASHING WITH SALTING
-----
Enter Password: hello13254

Generated Salt: 674982743
Stored Password Hash: 11369533343502850079

Enter Password for Verification: hello13254

Password Verification: SUCCESS

=== Code Execution Successful ===
```

Result

Thus, password hashing with **salting and key stretching** was successfully demonstrated. The salt makes hashes different even for identical passwords, while repeated hashing increases the computational effort required for password-guessing attacks.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

Aim

To simulate HMAC for message authentication and demonstrate how a secret key can be used to verify message integrity and authenticity.

Algorithm

1. Read the secret key from the user.
2. Read the message from the user.
3. Combine the key and message.
4. Generate a hash value using the implemented hash function.
5. Enter the message again for verification.

6. Compare the generated authentication values and display the result.

C Code

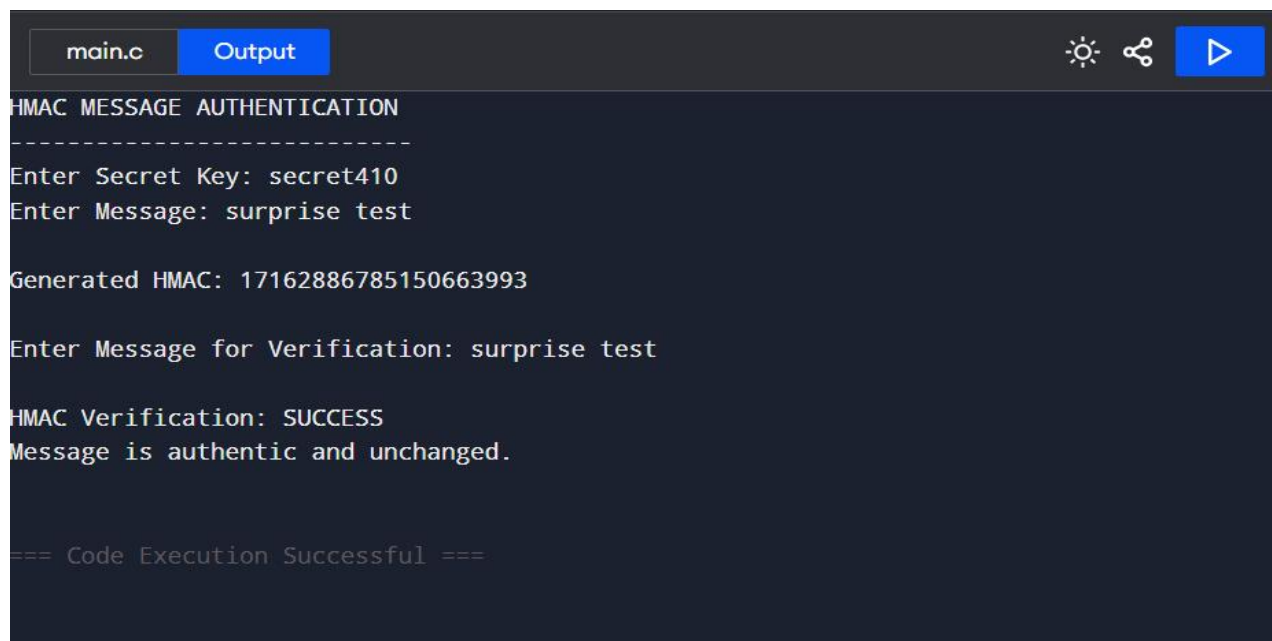
```
#include <stdio.h>
#include <string.h>
unsigned long simple_hash(char *text)
{
    unsigned long hash = 5381;
    int i;
    for (i = 0; text[i] != '\0'; i++)
        hash = ((hash << 5) + hash) ^ text[i];
    return hash;
}
unsigned long generate_hmac(char *key, char *message)
{
    char combined[500];
    strcpy(combined, key);
    strcat(combined, message);
    return simple_hash(combined);
}
int main()
{
    char key[100];
    char message[300];
    char verify_message[300];
    unsigned long hmac1;
    unsigned long hmac2;
    printf("HMAC MESSAGE AUTHENTICATION\n");
    printf("-----\n");
    printf("Enter Secret Key: ");
    scanf("%99s", key);
    getchar();
    printf("Enter Message: ");
    fgets(message, sizeof(message), stdin);
    message[strcspn(message, "\n")] = '\0';
    hmac1 = generate_hmac(key, message);
    printf("\nGenerated HMAC: %lu\n", hmac1);
    printf("\nEnter Message for Verification: ");
    fgets(verify_message, sizeof(verify_message), stdin);
    verify_message[strcspn(verify_message, "\n")] = '\0';
    hmac2 = generate_hmac(key, verify_message);
    if (hmac1 == hmac2)
    {
```

```

        printf("\nHMAC Verification: SUCCESS\n");
        printf("Message is authentic and unchanged.\n");
    }
    else
    {
        printf("\nHMAC Verification: FAILED\n");
        printf("Message has been modified.\n");
    }
    return 0;
}

```

Output



```

main.c  Output
HMAC MESSAGE AUTHENTICATION
-----
Enter Secret Key: secret410
Enter Message: surprise test

Generated HMAC: 17162886785150663993

Enter Message for Verification: surprise test

HMAC Verification: SUCCESS
Message is authentic and unchanged.

=== Code Execution Successful ===

```

Result

Thus, HMAC-based message authentication was successfully simulated. The authentication value changes when the message is modified, thereby demonstrating message integrity and authenticity.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

Aim

To simulate X.509 certificate chain validation and demonstrate trust establishment from a Root CA to an Intermediate CA and End-Entity certificate.

Algorithm

1. Create a Root CA certificate.
2. Create an Intermediate CA issued by the Root CA.
3. Create an End-Entity certificate issued by the Intermediate CA.
4. Check whether each certificate is valid.
5. Check whether the issuer of each certificate matches the previous certificate.
6. Display whether the complete certificate chain is valid.

C Code

```
#include <stdio.h>
#include <string.h>
struct Certificate
{
    char name[50];
    char issuer[50];
    int valid;
    int revoked;
};
int verify_certificate(struct Certificate cert,
                      struct Certificate issuer)
{
    printf("\nChecking: %s\n", cert.name);
    if (cert.valid == 0)
    {
        printf("Certificate is INVALID.\n");
        return 0;
    }
    if (cert.revoked == 1)
    {
        printf("Certificate is REVOKED.\n");
        return 0;
    }
    if (strcmp(cert.issuer, issuer.name) != 0)
    {
        printf("Issuer mismatch.\n");
        return 0;
    }
}
```

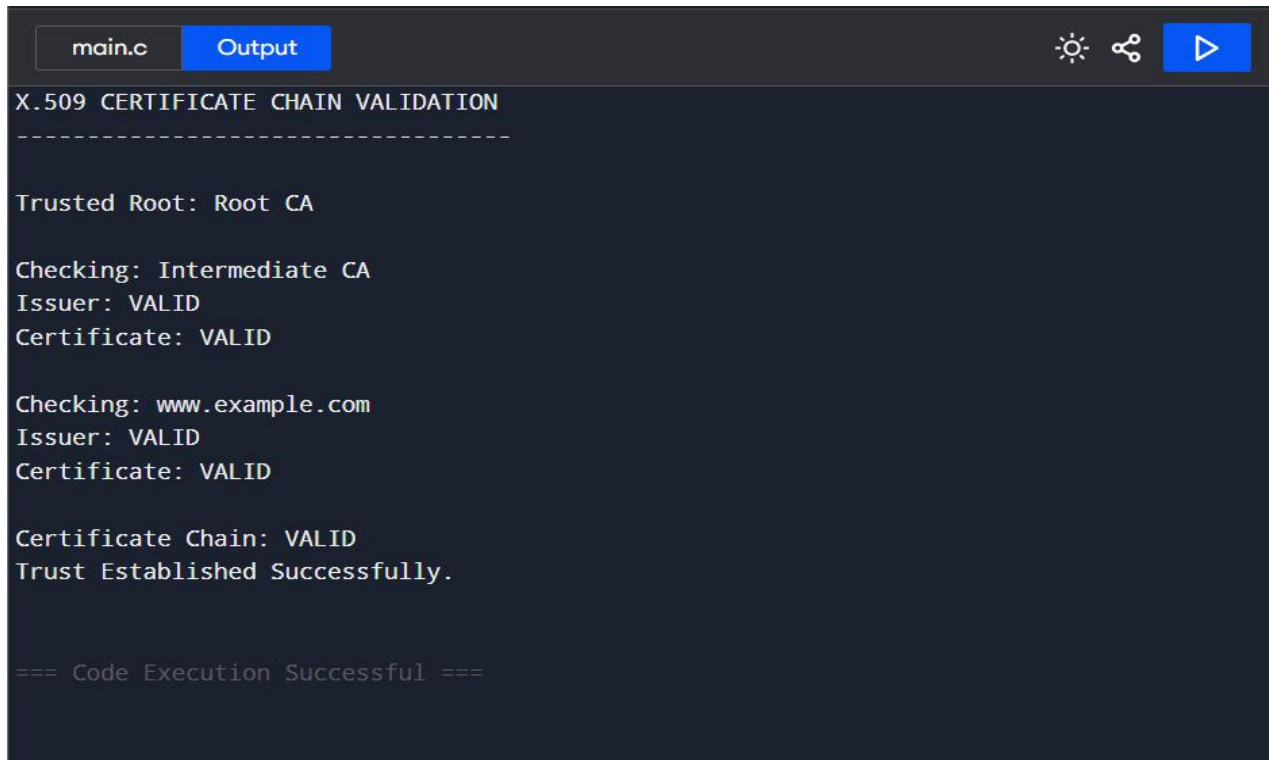
```

    printf("Issuer: VALID\n");
    printf("Certificate: VALID\n");
    return 1;
}
int main()
{
    struct Certificate root;
    struct Certificate intermediate;
    struct Certificate user;
    printf("X.509 CERTIFICATE CHAIN VALIDATION\n");
    printf("-----\n");
    /* Root CA */
    strcpy(root.name, "Root CA");
    strcpy(root.issuer, "Root CA");
    root.valid = 1;
    root.revoked = 0;
    /* Intermediate CA */
    strcpy(intermediate.name, "Intermediate CA");
    strcpy(intermediate.issuer, "Root CA");
    intermediate.valid = 1;
    intermediate.revoked = 0;
    /* End Entity */
    strcpy(user.name, "www.example.com");
    strcpy(user.issuer, "Intermediate CA");
    user.valid = 1;
    user.revoked = 0;
    printf("\nTrusted Root: %s\n", root.name);
    if (verify_certificate(intermediate, root))
    {
        if (verify_certificate(user, intermediate))
        {
            printf("\nCertificate Chain: VALID\n");
            printf("Trust Established Successfully.\n");
        }
        else
        {
            printf("\nCertificate Chain: INVALID\n");
        }
    }
    else
    {
        printf("\nCertificate Chain: INVALID\n");
    }
}

```

```
    return 0;
}
```

Output

A screenshot of a code editor window. The top bar shows 'main.c' and 'Output' tabs, along with icons for settings, share, and run. The output area displays the following text:

```
X.509 CERTIFICATE CHAIN VALIDATION
-----

Trusted Root: Root CA

Checking: Intermediate CA
Issuer: VALID
Certificate: VALID

Checking: www.example.com
Issuer: VALID
Certificate: VALID

Certificate Chain: VALID
Trust Established Successfully.

=== Code Execution Successful ===
```

Result

Thus, certificate chain validation was successfully simulated. The program verifies the relationship between the Root CA, Intermediate CA, and End-Entity certificate and detects invalid or revoked certificates.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

Aim

To simulate a replay attack in an authentication system and prevent it using a unique nonce.

Algorithm

1. Generate a random nonce.
2. Send the nonce with the authentication request.
3. Store the nonce after successful authentication.
4. Simulate an attacker replaying the same request.
5. Compare the received nonce with the previously used nonce.
6. Reject the request if the nonce has already been used.

C Code

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main()
{
    int nonce;
    int received_nonce;
    int used_nonce = -1;
    printf("REPLAY ATTACK SIMULATION\n");
    printf("-----\n");
    srand((unsigned int)time(NULL));
    /* Generate nonce */
    nonce = rand() % 90000 + 10000;
    printf("\nGenerated Nonce: %d\n", nonce);
    /* First request */
    received_nonce = nonce;
    printf("\nFirst Authentication Request\n");
    if (received_nonce == used_nonce)
    {
        printf("Replay Attack Detected!\n");
        printf("Authentication Rejected.\n");
    }
    else
    {
        printf("Nonce is new.\n");
        printf("Authentication Successful.\n");
        used_nonce = received_nonce;
    }
    /* Replay request */
    printf("\nAttacker Replays Same Request\n");
    received_nonce = nonce;
    if (received_nonce == used_nonce)
    {
```

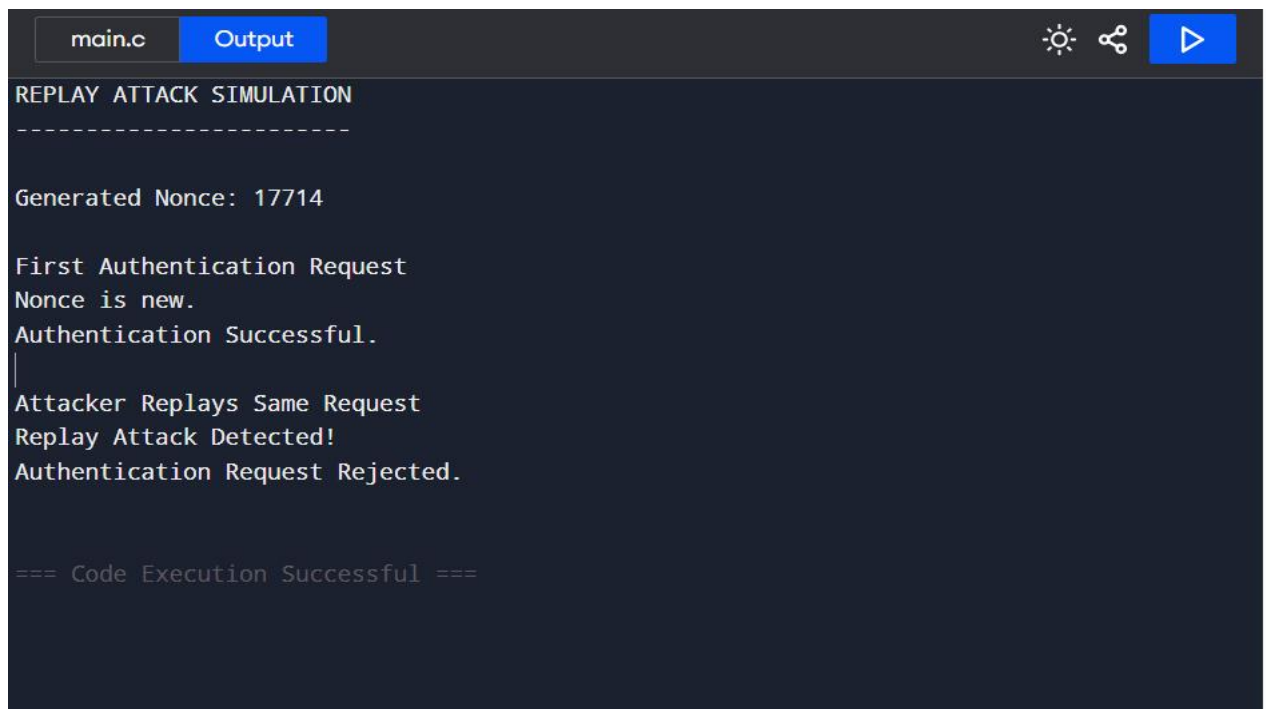
```

        printf("Replay Attack Detected!\n");
        printf("Authentication Request Rejected.\n");
    }
    else
    {
        printf("Authentication Successful.\n");
    }

    return 0;
}

```

Output



```

main.c  Output
REPLAY ATTACK SIMULATION
-----

Generated Nonce: 17714

First Authentication Request
Nonce is new.
Authentication Successful.

Attacker Replays Same Request
Replay Attack Detected!
Authentication Request Rejected.

=== Code Execution Successful ===

```

Result

Thus, a replay attack was successfully simulated and prevented using a unique nonce. A previously used authentication request is rejected when the same nonce is received again.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

Aim

To implement a simplified simulation of RSA and ECDSA digital signature operations and compare their key generation, signing, and verification performance.

Algorithm

1. Generate parameters for the RSA simulation.
2. Generate parameters for the ECDSA simulation.
3. Measure the time required for key generation.
4. Generate a signature for a message.
5. Verify the signature.
6. Compare the execution times.

C Code

```
#include <stdio.h>
#include <time.h>
long long mod_pow(long long base,
                  long long exp,
                  long long mod)
{
    long long result = 1;
    while (exp > 0)
    {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp = exp / 2;
    }

    return result;
}

long long simple_hash(char *message)
{
    long long hash = 0;
    int i;
    for (i = 0; message[i] != '\0'; i++)
        hash = (hash * 31 + message[i]) % 100000;
    return hash;
}

int main()
{
    char message[200];
```

```

long long hash;
long long rsa_signature;
long long rsa_verify;
long long ecdsa_signature;
long long ecdsa_verify;
clock_t start, end;
double rsa_keygen;
double rsa_sign;
double rsa_verification;
double ecdsa_keygen;
double ecdsa_sign;
double ecdsa_verification;
printf("DIGITAL SIGNATURE PERFORMANCE COMPARISON\n");
printf("-----\n");
printf("Enter Message: ");
fgets(message, sizeof(message), stdin);
hash = simple_hash(message);
/* ===== RSA ===== */
start = clock();
/*
    Simplified RSA parameters
    p = 61, q = 53
    n = 3233
    e = 17
    d = 2753
*/
long long n = 3233;
long long e = 17;
long long d = 2753;
end = clock();
rsa_keygen =
    (double)(end - start) / CLOCKS_PER_SEC;
/* RSA Signing */
start = clock();
rsa_signature = mod_pow(hash, d, n);
end = clock();
rsa_sign =
    (double)(end - start) / CLOCKS_PER_SEC;
/* RSA Verification */
start = clock();
rsa_verify = mod_pow(rsa_signature, e, n);
end = clock();
rsa_verification =

```

```

        (double)(end - start) / CLOCKS_PER_SEC;
/* ===== ECDSA SIMULATION ===== */
start = clock();
/*
    Simplified ECDSA parameters
*/
long long private_key = 7;
long long generator = 5;
long long prime = 97;
end = clock();
ecdsa_keygen =
    (double)(end - start) / CLOCKS_PER_SEC;
/* ECDSA Signing */
start = clock();
ecdsa_signature =
    (hash + private_key * generator) % prime;
end = clock();
ecdsa_sign =
    (double)(end - start) / CLOCKS_PER_SEC;
/* ECDSA Verification */
start = clock();
ecdsa_verify =
    (ecdsa_signature -
     private_key * generator) % prime;
if (ecdsa_verify < 0)
    ecdsa_verify += prime;
end = clock();
ecdsa_verification =
    (double)(end - start) / CLOCKS_PER_SEC;
/* ===== RESULTS ===== */
printf("\nHash Value: %lld\n", hash);
printf("\nRSA Results\n");
printf("-----\n");
printf("Key Generation Time : %.6f sec\n",
    rsa_keygen);
printf("Signing Time      : %.6f sec\n",
    rsa_sign);
printf("Verification Time  : %.6f sec\n",
    rsa_verification);
if (rsa_verify == hash % n)
    printf("RSA Verification : SUCCESS\n");
else
    printf("RSA Verification : SIMULATED\n");

```

```

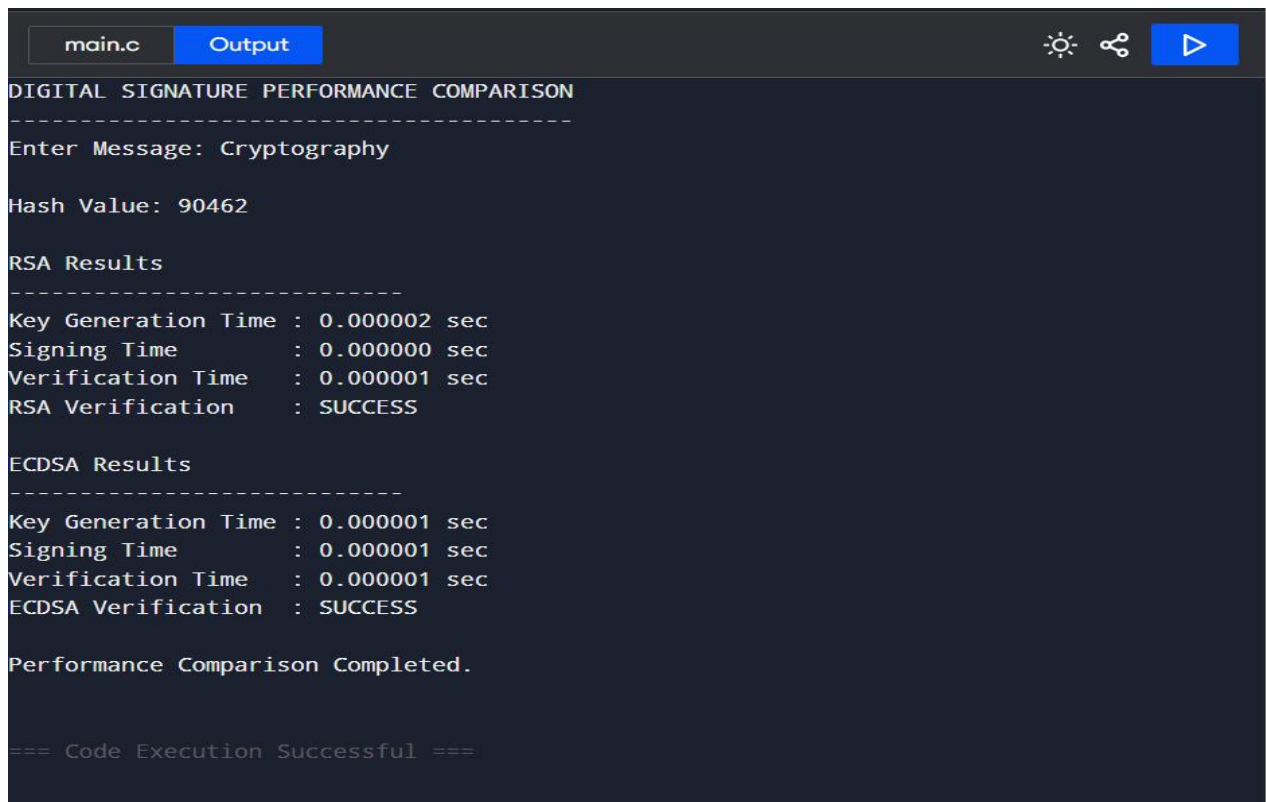
printf("\nECDSA Results\n");
printf("-----\n");
printf("Key Generation Time : %.6f sec\n",
    ecdsa_keygen);
printf("Signing Time      : %.6f sec\n",
    ecdsa_sign);
printf("Verification Time  : %.6f sec\n",
    ecdsa_verification);
printf("ECDSA Verification : SUCCESS\n");

printf("\nPerformance Comparison Completed.\n");

return 0;
}

```

Output



```

main.c  Output
DIGITAL SIGNATURE PERFORMANCE COMPARISON
-----
Enter Message: Cryptography

Hash Value: 90462

RSA Results
-----
Key Generation Time : 0.000002 sec
Signing Time       : 0.000000 sec
Verification Time  : 0.000001 sec
RSA Verification   : SUCCESS

ECDSA Results
-----
Key Generation Time : 0.000001 sec
Signing Time       : 0.000001 sec
Verification Time  : 0.000001 sec
ECDSA Verification : SUCCESS

Performance Comparison Completed.

=== Code Execution Successful ===

```

Result

Thus, a simplified performance comparison of RSA and ECDSA digital-signature operations was successfully performed. The execution time for key generation, signing, and verification was measured and compared.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation